Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de la Technologie

Université de Sfax | Faculté des Sciences de Sfax

Département d'Informatique | Master MRSI

Critical Review of Rapport 2

An Honest Assessment and Redirection Toward Rapport 4

Technical Report — Rapport-Critique

Prepared by

Ahmedou Yahye Kheyri

Supervised by

Pr. Hala Bezine

Academic Year 2025–2026 | April 2026

Abstract

This document is an internal critical review of Rapport 2 of the thesis *An Agentic Framework for Infrastructure as Code (IaC) Misconfiguration Analysis and Remediation*. It was written after a bibliographic survey of benchmarks and IaC security literature published between April 2025 and April 2026, and is intended to redirect the work toward Rapport 4 on a stronger empirical and positioning footing.

The review is deliberately blunt. Three of the six contributions claimed in Rapport 2 are reclassified as engineering deliverables or negative baselines rather than research contributions. Five of the eighteen evaluation metrics are either non-computable on the current backend, trivially saturated, or conceptually misapplied; concrete replacements are proposed. A benchmark landscape comparison places the framework's target niche — agentic IaC security remediation with independent tool-based validation — in context of recent benchmarks published in 2025 and early 2026 (PatchEval [1], AutoPatchBench [2], SEC-bench [3], SecRepoBench [4], IntelliSA [5], IaC-Eval [6], Multi-IaC-Eval [7]). The niche is confirmed to be genuinely unoccupied, but Rapport 2 has not yet demonstrated that the framework occupies it; Rapport 4 must close this gap.

Keywords: Self-critique, Research validity, IaC security, Benchmark survey, Rapport 4 planning.

Contents

Abstract **1**

1 Purpose and Scope **4**

1.1 Why this document exists 4

1.2 Review methodology 4

2 Claim-by-Claim Review of Rapport 2 Contributions **6**

2.1 Claim 1 — "Six-module architecture with iterative validation loop" 6

2.2 Claim 2 — "Labeled evaluation dataset of 27 smells / 8 files" 7

2.3 Claim 3 — "62-category taxonomy JSON" (`smells_taxonomy.json`) 8

2.4 Claim 4 — "Rigorous 18-metric evaluation methodology" 8

2.5 Claim 5 — "Preliminary experimental results: Macro-F1 = 0.082" 9

2.6 Claim 6 — "Phased implementation plan" 10

2.7 Summary of the re-graded contributions 11

3 Metrics Validity Audit **12**

3.1 Issue 1 — Log-probability confidence is not computable on Claude 12

3.2 Issue 2 — Hit Rate@5 = 1.0 is trivially saturated 13

3.3 Issue 3 — PVR is undefined on heuristic-only smells 13

3.4 Issue 4 — Fleiss' κ across runs is test–retest, not inter-rater 13

3.5 Issue 5 — Per-category F1 on $N = 1$ instances is noise 14

4 Benchmark Landscape (April 2025 – April 2026) **15**

4.1 Task × Domain grid 15

4.2 Interpretation 15

4.3 What to cite in Rapport 4 that is missing from Rapport 2 16

5 Technology Timeline for Rapport 4 **17**

5.1 Timeline skeleton (2019 – 2026) 18

5.2 TikZ rendering (to be polished for Rapport 4) 19

6 Redirection — What Rapport 4 Must Contain **20**

6.1 Proposed Rapport 4 structure 20
6.2 Code work to do before Rapport 4 20
6.3 Items deliberately left out 21

7 Summary **22**

Chapter 1

Purpose and Scope

1.1 Why this document exists

Rapport 2 was submitted in April 2026 as the main technical report of the thesis. It describes the framework architecture, a 27-smell evaluation dataset, eighteen evaluation metrics, and a Config A baseline (Macro-F1 = 0.082 for Checkov alone). Between Rapport 2 and the present date, the following things have happened:

- Rapport 3 has been drafted, documenting a 33,667-record IaC security fix dataset scraped from GitHub, Checkov, KICS, tfsec, and known vulnerable repositories.
- IntelliSA [5] has appeared on arXiv (January 2026). It benchmarks *Claude-4*, *Grok-4*, and *GPT-5* against GLITCH [8] on Puppet/Ansible/Chef with F1 of 0.77–0.88 — establishing a new SOTA in semantic IaC smell detection just weeks before this writing.
- PatchEval has been formally published at arXiv 2511.11019 with an end-to-end LLM patching benchmark of 1,000 CVEs and 65 CWE categories [1].
- Meta has released AutoPatchBench [2] (April 2025) for fuzzing-discovered vulnerabilities.
- SEC-bench [3] (June 2025) and SecRepoBench [4] (April 2025) have extended the LLM-based security repair benchmark ecosystem.
- Multi-IaC-Eval [7] (September 2025) has been released for CloudFormation/Terraform/CDK mutation tasks.

Rapport 2 does not cite any of these. That is not a defect in Rapport 2 itself — many of them were published after the Rapport 2 reading phase — but it means Rapport 4 must be written with the updated landscape in mind. This document is the bridge.

1.2 Review methodology

Each claim in Rapport 2 is re-examined along three axes:

1. **Originality**: is the claim a contribution in the research sense (new knowledge) or a deliverable (working artefact)?
2. **Empirical support**: does the rapport report numbers that back the claim, or is the claim stated without results?

3. **Comparability**: can the claim be situated in the current benchmark landscape?

Each claim is given a summary grade: **Keep** (defensible contribution), **Reframe** (defensible but miscategorised), **Demote** (not a research contribution).

Chapter 2

Claim-by-Claim Review of Rapport 2 Contributions

Rapport 2 lists six contributions in the Introduction (Chapter 1.4). Each is examined below.

2.1 Claim 1 — "Six-module architecture with iterative validation loop"

Grade: Reframe

Defensible at the level of *system integration* but not at the level of *architectural novelty*.

Every individual component has direct antecedents:

- **Iterative retrieval refinement on failure** is the defining feature of CRAG [9] and the active-query strategy of FLARE [10].
- **Self-critique and retry loops** are the core idea of SELF-REFINE [11], now widely adopted in agentic RAG systems [12].
- **Tool-based patch validation (before/after execution of a scanner or test suite)** is exactly the protocol used by PatchEval [1] and AutoPatchBench [2].
- **Confidence-based filtering of LLM output** is lifted from SecLLM [13] (Equation 1 of Rapport 2 is SecLLM's Equation 1 verbatim).

What is actually new is the *combination* applied to the IaC security domain: no prior system plumbs a scanner (Checkov) into the validation loop of a CRAG-style retriever and an LLM patcher *for IaC specifically*. But a combination is not a contribution unless it is shown to outperform the sum of its parts. Rapport 2 does not show this; only Config A has been run.

Redirection for Rapport 4: Reword this claim as *"the first integrated system that combines (RAG + LLM patching + IaC scanner validation + CRAG-style retry) for the IaC security domain"* and defer the contribution status until Config D (full pipeline) results are available.

2.2 Claim 2 — "Labeled evaluation dataset of 27 smells / 8 files"

Grade: Demote (and replace)

The 8-file corpus is a useful *demo set*; it is not a research dataset at thesis scale.

For reference:

Table 2.1: Scale comparison with comparable IaC smell datasets.

System	Files	Smell inst.	Source
GLITCH eval corpus	196,755	—	[8]
IntelliSA eval corpus	241	11,814 LOC	[5]
Rahman et al. (seven sins)	1,726	—	[14]
SecLLM eval (per technology)	400–500	—	[13]
Rapport 2 (this thesis)	**8**	**27**	Hand-annotated
Rapport 3 (this thesis, not yet cited in Rapport 2)	**33,667**	$\geq$ 50,000 (18 smell types)	GitHub + Checkov + KICS + tfsec

Finding

The 33,667-record dataset documented in Rapport 3 is the largest publicly available corpus of IaC security *fix pairs* in the literature. It is a real contribution. Rapport 2 does not mention it.

Redirection for Rapport 4:

1. Demote the 8-file, 27-smell set to *"a hand-curated sanity-check suite used for unit-level validation of the pipeline"*.
2. Promote the 33,667-record corpus (Rapport 3) as the dataset contribution of the thesis.
3. Report evaluation numbers on a *held-out test split* of the 33k corpus, not on the 8-file suite. This is consistent with IntelliSA [5], which reports on a held-out human-labelled subset, not on the files used to design the detector.

2.3 Claim 3 — "62-category taxonomy JSON" (`smells_taxonomy.json`)

Grade: Demote

Not a research contribution. This is a serialisation of War et al. [15] into JSON.

The taxonomy itself — the categories, CWE mapping, and cross-tool alignment — is the contribution of War et al. (2025). Encoding an existing taxonomy in machine-readable form is engineering; the novelty, if any, is in how the encoding is *used* (as a knowledge-base index), not in the encoding itself.

Redirection for Rapport 4: Do not list this as a contribution. Mention the file in the implementation chapter as part of the knowledge-base construction pipeline, and cite War et al. as the source of the taxonomy.

2.4 Claim 4 — "Rigorous 18-metric evaluation methodology"

Grade: Reframe

The methodology is sound, but two issues prevent it from being a contribution as written.

First, the count is inflated. Fifteen of the eighteen metrics are imported verbatim from prior work:

- Precision, Recall, F1, Macro-F1: classical, adopted in IaC by Rahman et al. [14], GLITCH [8], and SecLLM [13].
- Hit Rate@K, MRR: classical IR metrics [16].
- Context Recall, Context Precision: RAGAS [17].
- Confidence from token log-probabilities: SecLLM Equation 1.
- First-Attempt Success Rate, $\Delta R@k$: SELF-REFINE [11].
- Fleiss' Kappa: Fleiss et al. [18]; used in IaC by SecLLM.
- Wilcoxon Rank-Sum + Holm–Bonferroni correction: Wilcoxon [19] + Holm [20].

Second, fifteen of the eighteen metrics have *not been computed* in Rapport 2. They appear in the Evaluation Methodology chapter but are absent from the Results chapter. A methodology that is only applied to three metrics (Precision, Recall, F1 on Config A) is a *plan*, not an instantiation.

What is genuinely new:

- **Patch Validity Rate (PVR)** as formalised in Eq. 2 of Rapport 2 (intersection-and-regression criterion using a scanner as oracle). PatchEval [1] uses a similar idea for C/C++/Python test suites; as far as we can tell, PVR's formulation using a static IaC scanner as the oracle has not been published before.
- **Smell Elimination Rate (SER)** and **No-New-Issues Rate (NNIR)** as decompositions of PVR.

Redirection for Rapport 4: Claim PVR/SER/NNIR as the novel metric contribution; present everything else as *applied literature*. Drop the "18 metrics" framing and report the subset actually computed.

2.5 Claim 5 — "Preliminary experimental results: Macro-F1 = 0.082"

Grade: Demote

A low baseline due to known tooling artefacts is not a result.

Rapport 2 itself attributes the low Macro-F1 to two factors:

1. **Check ID version drift**: 9 of 16 False Negatives are caused by Checkov renaming check IDs between the version used to build the ground truth and the evaluation version (e.g., `CKV_AWS_52` → `CKV2_AWS_62`).
2. **Oracle scope mismatch**: the ground truth labels 27 smells, while Checkov 3.2.513 flags 83 issues. 76 of those 83 become False Positives by construction (the oracle simply never asked about them).

In other words, the $F_1 = 0.132$ and Macro-F1 = 0.082 numbers are not measuring Checkov's ability to detect real misconfigurations; they are measuring the mismatch between (a) a specific oracle scope and (b) a specific Checkov version. Both are artefacts of the evaluation setup, not properties of the framework.

Redirection for Rapport 4:

- Drop the Config A baseline in its current form from the contributions list.
- If re-reported, normalise it: compute precision/recall only against the *oracle-scoped* subset of Checkov outputs, not against the full 83.
- Replace the rapport-wide Macro-F1 target of 0.80 with a more realistic target: match IntelliSA's F1 of 0.77–0.88 on Puppet/ Ansible/Chef [5], adapted to the tools this thesis targets.

2.6 Claim 6 — "Phased implementation plan"

Grade: Demote

Project planning, not a research contribution.

A four-phase plan is a deliverable of the methodology section. It belongs in the implementation chapter or an appendix. It should not be in the contributions list of a research report.

2.7 Summary of the re-graded contributions

Table 2.2: Rapport 2 contributions after re-grading.

#	Original claim	Proposed treatment in Rapport 4	Grade
1	Six-module architecture	Reword as "first integration of RAG + LLM + IaC scanner + CRAG retry for IaC security", defer contribution status until Config D re-	Reframe

Chapter 3

Metrics Validity Audit

This chapter identifies metrics in Rapport 2 that, as specified, either cannot be computed on the intended backend, are trivially saturated on the current dataset, or are conceptually misapplied. Each is paired with a proposed fix.

3.1 Issue 1 — Log-probability confidence is not computable on Claude

Rapport 2, Equation 1:

$$\text{confidence} = \exp\left(\frac{1}{n}\sum_{i=1}^{n}\log p_i\right)$$

This is SecLLM's confidence score [13], which requires per-token log-probabilities from the LLM backend.

Backend constraints (verified April 2026):

- **OpenAI Chat Completions**: log-probabilities are returned only when `logprobs=true` is passed; the top-k token list is capped (typically at 20).
- **Anthropic Messages API (Claude)**: does not expose per-token log-probabilities at all as of the current API surface.

Since Rapport 2 lists *Claude Sonnet* as one of the supported backends, Eq. 1 is simply non-computable on half the intended deployment surface. This invalidates the confidence-filtering step and any metric derived from it (notably the Wilcoxon test of Section 2, Claim 4's Layer 3).

Proposed surrogate: self-consistency confidence — run the Fix Generator N times (e.g. $N = 5$) with temperature > 0, compute

$$\text{confidence}_{\text{self-cons}} = \frac{|\{i : \text{patch}_i \text{ is structurally identical to the modal patch}\}|}{N}$$

and threshold on that. This is the approach used in the Self-Consistency (SC) family of methods and is portable across backends.

3.2 Issue 2 — Hit Rate@5 = 1.0 is trivially saturated

Rapport 2 Table 7 reports Hit Rate@5 = 1.0 on 32 queries evaluated against the 62-entry taxonomy.

This is not surprising: the taxonomy was built around the same smell types that appear in the 27-smell dataset, so every query has a guaranteed match in the top few candidates. The metric is saturated by construction. Rapport 2 correctly flags this as a limitation but does not correct it.

Proposed fix:

- Use held-out queries derived from the 33k-record corpus (Rapport 3), where the query text (commit message or smell description from the scraper) is lexically different from the taxonomy entry.
- Report Hit Rate@K for $K \in \{1, 3, 5, 10\}$ on at least 1000 held-out queries, following the RAGAS evaluation protocol of Es et al. [17].

3.3 Issue 3 — PVR is undefined on heuristic-only smells

PVR, SER, and NNIR (Rapport 2 Equations 2, 4, 5) all require a scanner check ID on both the ground-truth smell and the post-patch state. Four of the 27 ground-truth smells in Rapport 2 are labelled `HEURISTIC` (primarily Ansible smells not covered by Checkov). For these four, S_{target} is undefined, and so is PVR.

Proposed fix: add a second validator so every ground-truth smell has at least one scanner that can witness it. Two options:

1. **tfsec**: fast, Terraform-focused. Not useful for Ansible.
2. **KICS (Checkmarx)**: covers 22+ platforms including Ansible, Kubernetes, and Dockerfile with 2,400+ queries. Strongly recommended as the second validator.

KICS is already ingested in the 33k-record corpus (1,336 records come from KICS test resources) and is the natural choice.

3.4 Issue 4 — Fleiss' κ across runs is test–retest, not inter-rater

Rapport 2 Layer 4 proposes to report Fleiss' Kappa across 5 LLM runs and interpret it using the Landis & Koch scale [21].

This is a misapplication. Fleiss' Kappa is designed to measure *inter-rater agreement*: k distinct raters independently judging the same items. Re-running the same LLM is not "five raters"; it is *one rater measured* 5 *times*. The correct quantity is **test–retest reliability** (also expressible as Cohen's Kappa between pairs of runs or as intra-class correlation).

SecLLM [13] reports $\kappa = 1.00$, which is trivial because they cache LLM responses between runs — so the five "raters" are literally reading from the same cache. Any $\kappa < 1$ in the Rapport 4 experiments should be interpreted as LLM non-determinism, not as disagreement between judges.

Proposed fix: rename the metric in Rapport 4 to *Test–Retest Agreement* (or simply *stability across runs*), and report it as Cohen's κ between consecutive runs or as the standard deviation of PVR across runs, not as Fleiss' κ.

3.5 Issue 5 — Per-category F1 on $N = 1$ instances is noise

Rapport 2 Table 3 reports the evaluation dataset composition. Several smell categories have only 1 or 2 instances: CWE-295, CWE-829, CWE-693, CWE-400. Macro-F1 weights these categories equally with CWE-732 (8 instances) and CWE-250 (8 instances). On a single-instance category, F1 is 0, 1, or undefined — not a usable signal.

Proposed fix:

- Pool rare categories under a *"Other"* bucket with ≥ 5 instances before computing Macro-F1.
- Switch to micro-F1 as the primary aggregate on the 33k corpus (where category counts are > 100 each).
- Keep per-category reporting only for categories with ≥ 10 instances.

Chapter 4

Benchmark Landscape (April 2025 – April 2026)

This chapter surveys benchmarks *relevant to the thesis target* that have appeared in the twelve months preceding this writing. The goal is to situate the framework's contribution precisely — not as "the first X for IaC", but as the first X at a specific position on the (*task* × *domain*) grid.

4.1 Task × Domain grid

Table 4.1: Benchmark landscape, April 2025 – April 2026.

Benchmark	Task × Domain	Cite
IaC-Eval	IaC *generation* × Terraform/AWS (458 scenarios; GPT-4 pass@1 = 19.4%)	[6]
Multi-IaC-Eval	IaC *mutation* × CF/TF/CDK (synthetic triples with LLM validation)	[7]
IntelliSA	IaC *smell detection* × Puppet/Ansible/Chef (symbolic + neural; F1 = 0.77–0.88 with Claude-4/Grok-4/GPT-5)	[5]
SecLLM	IaC *smell detection* × Ansible/Chef/Puppet (prompt-engineered GPT-4o-mini/Qwen-2.5; F1 up to 0.90)	[13]
GenSIAC	IaC *secure generation* × multi-tool (instruction-tuning; F1 0.30 → 0.85)	[22]
PatchEval	Patch *generation* × 1000 CVEs (Python/Go/JS; 65 CWEs; dynamic sandbox + static similarity)	[1]
AutoPatchBench	Patch *generation* × fuzzing-found vulns in C/C++ (Meta internal)	[2]
SEC-bench	Agent-built CVE benchmark × multi-language (LLM agents construct the benchmark itself)	[3]
SecRepoBench	Secure *code generation* × real-world repo functions	[4]
This thesis (target)	**Agentic IaC *remediation* × TF/K8s/Ansible/Docker/CF** with **independent scanner validation (PVR)**	—

4.2 Interpretation

Two observations follow from Table 4.1:

1. The *IaC* × *smell-detection* cell is crowded and maturing fast. IntelliSA [5] (Jan 2026) and SecLLM [13] (2025) both report F1 above 0.77 using state-of-the-art LLMs. Any detection-only contribution by this thesis will need to match or exceed these numbers, on the same evaluation set, to be competitive.

2. The *IaC* × *remediation + tool-validated* cell is empty. PatchEval and AutoPatchBench validate patches against test suites (dynamic) for general code; no IaC-specific remediation benchmark exists. This is the thesis's niche, and it is genuinely unoccupied.

Positioning for Rapport 4

The thesis contribution must be framed as **remediation-and-validation**, not detection. Detection-side comparisons should use IntelliSA as the SOTA baseline; remediation-side PVR/SER/NNIR numbers are the *novel* experimental contribution because no prior benchmark computes them for IaC.

4.3 What to cite in Rapport 4 that is missing from Rapport 2

- IntelliSA [5] (Jan 2026) — SOTA on IaC smell detection; cite in the State of the Art chapter and use as the detection-side baseline.
- PatchEval [1] (Nov 2025) — the closest methodological precursor to PVR; cite when defining PVR.
- AutoPatchBench [2] (Apr 2025, Meta) — tool-validated patch evaluation at industrial scale; cite alongside PatchEval.
- SEC-bench [3] (Jun 2025) — agent-built benchmark construction, relevant to the scraping pipeline (Rapport 3).
- SecRepoBench [4] (Apr 2025) — repository-scale secure code generation.
- IaC-Eval [6] (NeurIPS 2024) — IaC generation baseline; useful to show that even generation is hard for current LLMs (GPT-4 pass@1 = 19.4%).
- Multi-IaC-Eval [7] (Sept 2025) — IaC mutation across CF/TF/CDK; relevant because our framework also performs a mutation (insecure → secure).
- Agentic RAG survey [12] — frames the CRAG retry loop in the current agentic-RAG taxonomy.

Chapter 5

Technology Timeline for Rapport 4

Rapport 4 should open with a timeline chapter placing the project in the evolution of RAG, LLMs for code, and IaC security tooling. A skeleton is given below and a TikZ rendering is provided for use in Rapport 4.

5.1 Timeline skeleton (2019 – 2026)

Table 5.1: Technology timeline relevant to the thesis.

Year	Track	Milestone	Relevance
2019	IaC taxonomy	Rahman "Seven Sins" [14]	Foundational smell vocabulary
2020	RAG	Lewis et al. "Retrieval-Augmented Generation" (NeurIPS)	First RAG paper (Naive RAG)
2020	IaC taxonomy	Rahman "Gang of Eight" [23]	Extended taxonomy
2022	IaC static	GLITCH [8] (ASE 2022)	Polyglot IR-based detection
2023 Feb	Tool-use	Toolformer [24]	LLMs calling external tools
2023 Mar	Self-correction	SELF-REFINE [11]	Iterative self-feedback
2023 May	Active RAG	FLARE [10]	Query refinement on failure
2023 Oct	Self-RAG	Asai et al. [25] (ICLR 2024)	LLM decides when to retrieve
2024 Jan	CRAG	Yao et al. [9]	Corrective

5.2 TikZ rendering (to be polished for Rapport 4)

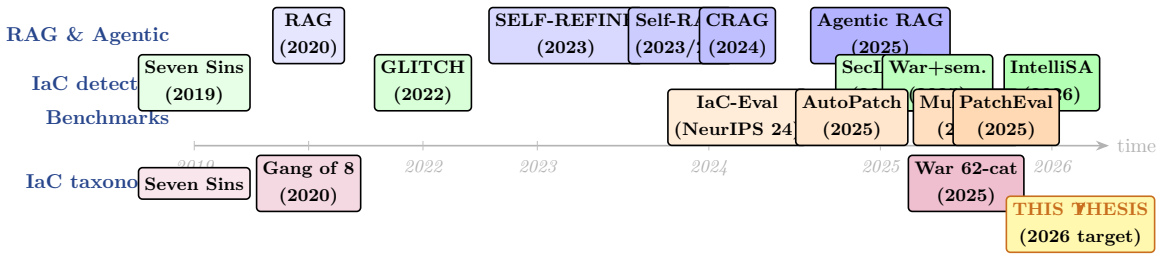

Figure 5.1: Technology-evolution timeline across four tracks relevant to the thesis. The thesis target sits at the intersection of all four — a remediation contribution that uses the 2024-era CRAG retry pattern, the 2025 War 62-category taxonomy, and the 2025 PatchEval-style scanner-validated evaluation, applied to the IaC domain where IntelliSA now sets the 2026 detection baseline.

Chapter 6

Redirection — What Rapport 4 Must Contain

This chapter is the deliverable intent: a blueprint for Rapport 4 and for the code work that precedes it.

6.1 Proposed Rapport 4 structure

1. **Technology Timeline and Positioning** — the timeline of Chapter 5 and the landscape grid of Chapter 4, with the thesis niche identified.
2. **Updated State of the Art** — add IntelliSA [5], PatchEval [1], AutoPatchBench [2], SEC-bench [3], SecRepoBench [4], IaC-Eval [6], Multi-IaC-Eval [7], agentic-RAG survey [12].
3. **Framework (carried over from Rapport 2)** — no major redesign; minor updates to reflect the KICS second-validator decision and the self-consistency confidence surrogate.
4. **Dataset** — present the 33,667-record corpus (Rapport 3) as the primary evaluation dataset; the 8-file suite becomes a sanity-check appendix.
5. **Evaluation Methodology** — keep PVR/SER/NNIR; drop the log-prob confidence metric; rename the test–retest kappa; reduce the "18 metrics" count to the actually-computed subset.
6. **Experimental Results** — *the critical chapter*; must report Config B, C, D on a held-out test split of the 33k corpus, with PVR as the headline metric and a direct comparison to IntelliSA on the detection layer.
7. **Threats to Validity** — acknowledge LLM non-determinism and data leakage; add the KICS/Checkov coverage gap for remaining HEURISTIC-only smells.
8. **Conclusion and Perspectives**.

6.2 Code work to do before Rapport 4

Ordered by priority:

1. **Finish Phase 4 of the scraping plan** — Checkov validation on the 33k corpus is currently running (`Code/scraping/plan.md`). Needed to populate Tier A.
2. **Add KICS as a second validator** in `Code/src/validator/tool_integrator.py`. KICS covers Ansible/K8s/Docker/CF/Terraform, unlike Checkov's partial Ansible support. Closes Issue 3 of Chapter 3.
3. **Replace log-prob confidence** in `Code/src/generator/fix_generator.py` with self-consistency confidence over $N = 5$ sampled patches. Closes Issue 1 of Chapter 3.
4. **Wire the 33k corpus into** `Code/scripts/evaluate.py` as the primary evaluation dataset. Keep the 8-file suite as a CI-level unit test.
5. **Run Config B, C, D end-to-end** on the held-out test split (3,371 records). Collect PVR, SER, NNIR, and the detection-layer P/R/F1 comparable to IntelliSA.
6. **Reproduce the SOTA baseline** — run IntelliSA (or its nearest public implementation) on the same held-out test split so the comparison in Rapport 4 is numerical, not indirect.

6.3 Items deliberately left out

- The log-prob confidence Wilcoxon analysis. Not worth salvaging on Claude.
- The 18-metric framing. Rapport 4 should report the metrics that are actually computed, and no others.
- OSV/NVD-based Tier A enrichment — already determined to be low-yield (see the project memory).

Chapter 7

Summary

Bottom line

Rapport 2 is a solid *design* document and a *plan*. It is not yet a *results* document. Three of its six claimed contributions do not survive a careful reading; the other three are defensible but need empirical backing that does not yet exist. The single most important missing piece is the Rapport 3 dataset, which is larger than any comparable IaC smell corpus and should be promoted to the primary contribution of the thesis. The second most important missing piece is a Config D run — the actual experiment that would justify the framework's existence. Rapport 4 must deliver both.

The niche is real; the tools to fill it are partly written; the benchmark landscape has a hole exactly where the thesis aims. What remains is to run the experiments, report them honestly, and let the PVR numbers speak.

Bibliography

[1] X. Lyu, S. Wang, C. Li, and C. Liu, "PatchEval: A benchmark for evaluating LLMs on patching real-world vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2511.11019

[2] Meta Platforms, Inc., "AutoPatchBench: A benchmark for ai-powered security fixes," https://engineering.fb.com/2025/04/29/ai-research/autopatchbench-benchmark-ai-powered-security-fixes/, 2025, engineering at Meta blog, 29 April 2025.

[3] A. authors, "SEC-bench: Automated benchmarking of LLM agents on real-world software security vulnerabilities," 2025. [Online]. Available: https://arxiv.org/abs/2506.11791

[4] ——, "SecRepoBench: Benchmarking LLMs for secure code generation in real-world repositories," 2025. [Online]. Available: https://arxiv.org/abs/2504.21205

[5] ——, "IntelliSA: An intelligent static analyzer for IaC security smell detection using symbolic rules and neural inference," 2026. [Online]. Available: https://arxiv.org/abs/2601.14595

[6] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. Park, G. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, "IaC-Eval: A code generation benchmark for cloud infrastructure-as-code programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. [Online]. Available: https://openreview.net/forum?id=7TCK0aBL1C

[7] A. authors, "Multi-IaC-Eval: Benchmarking cloud infrastructure as code across multiple formats," 2025. [Online]. Available: https://arxiv.org/abs/2509.05303

[8] N. Saavedra and J. a. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE/ACM, 2022, pp. 1–12.

[9] S.-Q. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "CRAG: Comprehensive retrieval augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[10] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.06983

[11] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck,

A. Yazdanbakhsh, and P. Clark, "SELF-REFINE: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[12] A. authors, "Towards agentic RAG with deep reasoning: A survey of RAG-reasoning systems in LLMs," 2025. [Online]. Available: https://arxiv.org/abs/2507.09477

[13] G. De Vito, F. Palomba, and F. Ferrucci, "SecLLM: Enhancing security smell detection in IaC with large language models," *IEEE Access*, vol. 13, pp. 204 480–204 498, 2025.

[14] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 164–175.

[15] A. War, S. L. B. Nikiema, J. Samhi, J. Klein, and T. F. Bissyande, "Security smells in infrastructure as code scripts: A taxonomy update beyond the seven sins," 2025. [Online]. Available: https://arxiv.org/abs/2511.12884

[16] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/

[17] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024. [Online]. Available: https://arxiv.org/abs/2309.15217

[18] J. L. Fleiss, B. Levin, and M. C. Paik, *Statistical Methods for Rates and Proportions*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[19] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[20] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[21] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[22] Y. Li, M. Grella, D. Nahmias, G. Engelberg, D. Klein, G. Guizzardi, T. van Ede, and A. Continella, "GenSIAC: Toward security-aware infrastructure-as-code generation with large language models," 2025. [Online]. Available: https://arxiv.org/abs/2505.01568

[23] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering (ICSE)*, 2020, pp. 752–764.

[24] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[25] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[26] D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," 2025. [Online]. Available: https://arxiv.org/abs/2506.18795

[27] A. War, A. N. Rawass, A. K. Kabore, J. Samhi, J. Klein, and T. F. Bissyande, "Detection of security smells in IaC scripts through semantics-aware code and language processing," 2025. [Online]. Available: https://arxiv.org/abs/2509.18790